Parallel File Search Using Threads

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATIONAL TECHNOLOGY

By

Shanmukha	**2520090050**
L. Sainath	**2520090148**
R. Nishanth	**2520090141**

Under the Esteemed Guidance of

D. Alekhya
Faculty Guide
Department of Computer Science and Informational Technology



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled “Parallel File Search Using Threads” is a record of bonafide work carried out by Shanmukha (2520090050), L. Sainath (2520090148) and R. Nishanth (2520090141) as part of the Operating Systems Hackathon project at K L (Deemed to be) University.
The implementation demonstrates concurrent file searching in C using POSIX threads on Linux, including thread synchronization, file handling, and execution-time comparison. The report is submitted in partial fulfillment of the requirements of the Bachelor of Technology programme in Computer Science and Engineering.

Shanmukha **2520090050**

L. Sainath **2520090148**

R. Nishanth **2520090141**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the Operating Systems hackathon project report entitled “Parallel File Search Using Threads” was completed by Shanmukha (2520090050), L. Sainath (2520090148) and R. Nishanth (2520090141) in partial fulfillment of the Bachelor of Technology degree requirements in Computer Science and Engineering at K L (Deemed to be) University during academic year 2026–2027.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty members and the Department of Computer Science and Engineering for providing the opportunity to work on this Operating Systems hackathon project.

We are thankful to our faculty guide and course coordinator for their guidance, feedback, and encouragement throughout the development and testing of the project.

We also acknowledge the open-source GNU Compiler Collection, Linux system libraries, and POSIX thread interfaces used to implement and validate the application.

Finally, we thank our classmates, friends, and family for their support during the project work.

Shanmukha **2520090050**

L. Sainath **2520090148**

R. Nishanth **2520090141**

Abstract

Searching for a keyword across a large collection of text files is a common task in log inspection, source-code review, and document retrieval. A simple sequential approach opens and scans each file one after another. As the number or size of files increases, the total work can grow, and a single execution flow cannot make use of multiple CPU cores for independent file-processing tasks.

This project implements Parallel File Search Using Threads in C on Ubuntu/Linux. The program collects regular files directly inside a selected directory, accepts a search keyword and worker-thread count, and searches file contents line by line. A sequential baseline is executed first, followed by a parallel phase using POSIX threads. Each worker claims the next file index under a mutex; result printing is protected by a second mutex; each thread maintains local counters that the main thread combines after pthread_join completes.

The application reports matching filenames, line numbers, matching text, matching-file and matching-line totals, elapsed time, and the observed sequential-to-parallel speedup. Tests confirm that the sequential and parallel result counts agree for the tested datasets. In one recorded run over 300 synthetic files containing approximately 105,000 lines, both modes found 12 matching files and 24 matching lines, with an observed 3.01× speedup; this is a single-run measurement and is not a guarantee for every machine or workload. The project demonstrates practical operating-system concepts including multithreading, shared-state synchronization, file I/O, memory management, and performance evaluation.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		9
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		10
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		11
		4.1 C Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		12-13
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		14-15
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		16-17

		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Testing, Results and Impact**		**18**
		8.1 Testing Methodology	
		8.2 Test Cases and Outcomes	
		8.3 Results Achieved	
		8.4 Performance Interpretation	
9	**Conclusion and References**		**19**
		9.1 Summary of the Project	
		9.2 Objectives Achieved	
		9.3 Skills and Learning Outcomes	
		9.4 References	
	Appendix A-B	Source code and reproducible test commands	20-24

CHAPTER 1 - INTRODUCTION

1.1 Background of the Project

Text search is a fundamental operation in system administration and software development. Developers and administrators frequently scan logs, configuration files, reports, and source files to locate a warning, error message, identifier, or phrase. A basic search can be performed by opening each file and examining its lines. This is easy to understand but processes the file list sequentially.

Modern computers can run multiple threads, and independent files are natural units of work: one worker can search one file while another worker searches a different file. A threaded implementation must still coordinate shared work assignment and output, and performance should be measured instead of assumed.

1.2 Problem Statement

Design and implement a Linux command-line program that searches for a supplied keyword across regular text files in a directory. The program must report the matching filename and line number, support a configurable number of POSIX worker threads, protect shared state from race conditions, and compare sequential and parallel execution results and elapsed time.

1.3 Scope of the Project

• Searches regular files located directly inside the specified directory; nested subdirectories are not traversed in this version.

• Uses case-insensitive substring matching, so “error”, “ERROR”, and “Error” match the same keyword.

• Reads files as text using fgets() and reports a filename, line number, and matching line for every match.

• Runs a sequential baseline and a multithreaded search within one program execution.

• Provides command-line input rather than a graphical user interface or network service.

• Measures time with CLOCK_MONOTONIC and reports an observed speedup ratio.

1.4 Objectives of the Project

• Demonstrate thread creation and completion using pthread_create() and pthread_join().

• Distribute file-search tasks among a configurable number of worker threads.

• Use mutex locks to make task allocation and console output thread-safe.

• Keep per-thread counters local and aggregate them after all workers finish.

• Validate parallel-search correctness by comparing matching file and line counts with the sequential baseline.

• Measure performance on small sample files and a larger synthetic dataset, documenting the limitations of one-run measurements.

1.5 Importance of the Application

The project connects operating-system theory with a practical utility. It makes visible the trade-off between concurrency benefits and thread-management overhead, and demonstrates why shared data requires explicit synchronization. The same principles can be applied to log analysis, repository scanning, batch text processing, and other tasks that can be divided into independent units of work.

1.6 Target Users / Audience

The intended audience includes Operating Systems students, faculty evaluators, beginner Linux users, and developers who need to understand thread coordination and performance comparison. The code is intentionally command-line based so the main concepts can be observed using a small and reproducible dataset.

CHAPTER 2 - OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

A thread is an execution path within a process. Threads in the same process share the address space and process resources but maintain separate execution state, such as a stack and register context. POSIX threads (pthreads) expose thread creation, joining, and mutex synchronization APIs on Linux.

This implementation uses a main thread to prepare the search and coordinate completion, plus worker threads for concurrent file scanning. Each worker reads a claimed file independently. The thread pool is created once for the parallel phase and each worker repeatedly claims the next remaining file until the shared index reaches the file count.

2.1.1 Race Conditions and Mutexes

A race condition can occur when concurrent threads read or modify shared data without appropriate coordination. In this project, next_file is shared: without a mutex, two workers could claim the same file or update the index inconsistently. task_mutex protects the check-and-increment operation. output_mutex protects complete output messages so lines from different workers do not interleave at the character level.

The matching counters are stored separately in each ThreadData structure. A worker updates only its own counters, and the main thread aggregates them after joining the workers. This reduces the amount of shared mutable state and avoids locking a global counter for every match.

2.1.2 File I/O and Timing

The program uses opendir() and readdir() to enumerate directory entries, stat() to identify regular files, fopen() to open a file, fgets() to read lines, and fclose() to close it. clock_gettime(CLOCK_MONOTONIC) measures elapsed time using a monotonic clock suitable for duration measurements. strcasestr() performs a case-insensitive substring search and is available through the GNU C library when _GNU_SOURCE is defined.

2.2 Existing Methods / Algorithms

Method	Operation	Strength	Limitation
Sequential scan	Open each file and read all lines before moving to the next file.	Simple control flow; low coordination overhead.	Only one file is processed at a time.
One thread per file	Create a separate thread for every file.	Direct mapping between files and workers.	Can create too many threads and consume excessive resources.
Fixed worker pool / shared index	Create a bounded number of workers; each claims a file from shared work state.	Limits concurrency and balances uneven file sizes.	Requires synchronized task assignment and joining.

The proposed implementation uses the third approach. File sizes may vary, so workers claim work dynamically from a shared next_file index rather than receiving a fixed contiguous range. The critical section is short: it protects only the file-index claim, not the potentially long file scan.

2.3 Proposed Approach

- Parse the directory path, keyword, and requested worker count from command-line arguments.
- Collect all regular files directly inside the directory into a dynamically allocated list.
- Run a sequential pass and record matching-file count, matching-line count, and elapsed time.
- Reset the task index, create worker threads, and let each worker claim one file at a time under task_mutex.
- Search each claimed file without holding the task mutex; print matches while holding output_mutex.
- Join every worker, aggregate each worker's local counters, compare totals, compute speedup, and release resources.

2.4 Advantages of the Proposed Approach

The implementation uses a bounded, configurable worker count, dynamically assigns files to workers, protects shared task allocation and output, and aggregates private counters after joins. It checks correctness by comparing sequential and parallel totals and reports an observed speedup rather than assuming that concurrency is always faster.

CHAPTER 3 - SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The application has modest hardware requirements because it performs text reading and string matching without a graphical interface. A standard laptop or desktop with a working Linux installation is sufficient. Multiple CPU cores may help for CPU-bound workloads, but they are not a prerequisite for correctness.

Component	Recommended / minimum requirement
Processor	Any modern 64-bit CPU; multiple cores recommended for parallel experiments.
Memory	At least 2 GB RAM; more memory may improve caching for larger datasets.
Storage	A few hundred megabytes for source code and test files; larger datasets require additional space.
Display / terminal	Terminal emulator capable of displaying command-line output.

3.2 Software Requirements

Software	Purpose
Ubuntu / Linux	Operating environment providing the GNU C library and POSIX thread APIs.
GCC / build-essential	Compiles the C source file into an executable.
POSIX threads (pthreads)	Provides pthread_create(), pthread_join(), and pthread mutex operations.
GNU C library	Provides file, directory, timing, and case-insensitive string functions used by the program.
Terminal and text editor	Used to create the source file, compile, run, and inspect results.

3.3 Operating System Environment - Ubuntu / WSL

The intended environment is Ubuntu Linux. Windows users may also use Ubuntu under WSL if the compiler and POSIX thread runtime are installed. The development workflow is terminal-based and does not require a code editor or graphical package.

Install the compiler toolchain with:

```
sudo apt update
sudo apt install build-essential
```

3.4 Development Tools and Compilers

The project was compiled with GCC using warnings enabled and pthread support:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 parallel_search.c -o parallel_search -pthread
```

The -Wall, -Wextra, and -Wpedantic options ask the compiler to report common issues; -O2 enables optimization; and -pthread tells GCC to compile and link using POSIX thread support. The source defines _GNU_SOURCE to declare the GNU-specific strcasestr() function.

CHAPTER 4 - TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C is used because it exposes the APIs and resource-management steps commonly taught in an Operating Systems course. The program uses structures for per-thread statistics, dynamically allocated arrays for filenames, and functions to separate directory discovery, searching, worker logic, and result aggregation.

4.2 GCC Compiler

GCC compiles the source code into a native Linux executable. Compiler warnings are enabled during the build so the implementation can be inspected for type mismatches, unused declarations, and other common issues. The compiled binary can be run repeatedly against different directories, keywords, and worker counts.

4.3 Linux File and Directory APIs

opendir() opens a directory stream and readdir() returns its entries. stat() provides file metadata that is used to accept regular files. fopen(), fgets(), and fclose() perform line-oriented text-file reading. These APIs make the project a practical example of user-space interaction with the Linux file system through the standard C library.

4.4 POSIX Threads and Synchronization

pthread_create() starts a worker thread, pthread_join() waits for it to finish, and pthread_mutex_lock()/pthread_mutex_unlock() protect shared operations. The task mutex is used only while choosing a file index; the output mutex is used only while printing a complete matching line or reporting file I/O errors.

4.5 Linux Commands and Utilities

- mkdir -p: creates directories for source data and test fixtures.
- nano: edits the C source directly in a terminal.
- gcc: compiles the program and links thread support.
- ls -lh: verifies that the executable was created.
- ./parallel_search: launches the program with a directory, keyword, and thread count.
- time or the program's own clock_gettime measurements: helps compare elapsed duration.

4.6 Other Libraries / Functions

The implementation uses clock_gettime(CLOCK_MONOTONIC) for duration measurements, strcasestr() for case-insensitive substring matching, and malloc-family functions (realloc(), strdup(), and free()) for memory management. It relies on the system C library and requires no external package dependencies.

CHAPTER 5 - SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application is divided into a main coordinator, a directory scanner, a file-list data structure, a sequential search routine, a parallel worker routine, shared synchronization primitives, and a result aggregator. The same search_file() function is called from the sequential baseline and from worker threads so the matching logic remains consistent.

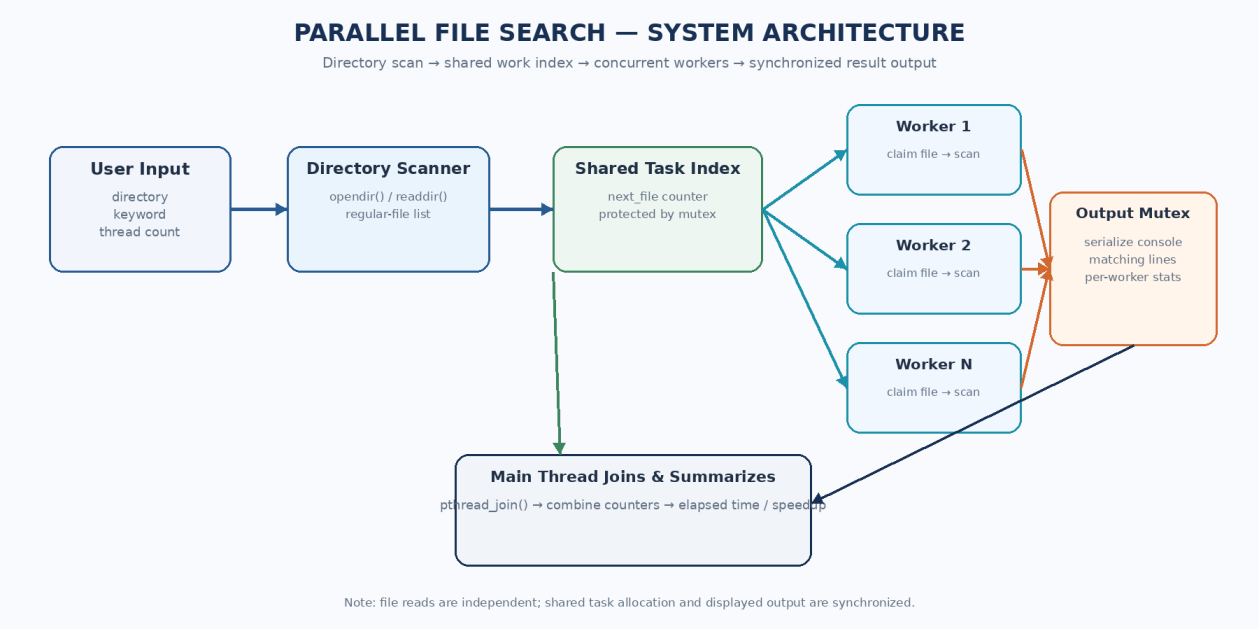


Figure 5.1. System architecture of the parallel file search application.

5.2 System Workflow

The user provides three command-line arguments: directory path, search keyword, and worker count. After validating the arguments, the program enumerates regular files and stores their paths. It then performs a sequential search, measures elapsed time, starts the worker threads, joins them, compares counters, calculates speedup, and releases all allocated memory.

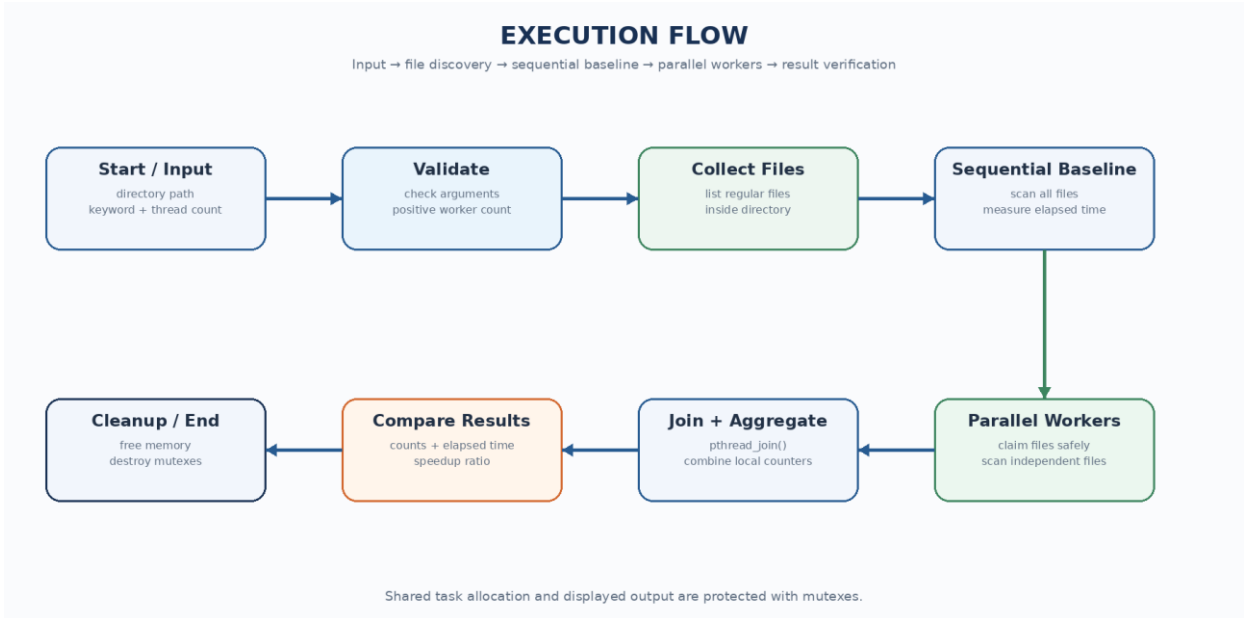


Figure 5.2. Control-flow diagram from input validation through result comparison.

5.3 Flowchart / Data Flow

The task index is the central shared work-distribution mechanism. A worker locks task_mutex, checks whether next_file is less than file_count, claims the current index, increments next_file, and releases the mutex. It then scans the file outside the critical section. If a line matches, the worker locks output_mutex while printing the result. Each thread updates its own SearchStats structure; the main thread combines those structures only after all joins complete.

5.4 Module Description

Module / function	Responsibility	Key OS concept
collect_files()	Enumerates regular files and builds a dynamic path list.	Directory and file metadata APIs.
search_file()	Reads one file line by line and tests each line against the keyword.	File I/O and string search.
sequential_search()	Runs the shared search function one file at a time.	Baseline measurement.
worker()	Claims file indices safely and searches files independently.	Thread execution and mutex synchronization.
parallel_search()	Creates workers, joins them, and merges per-thread counters.	Concurrency lifecycle.
main()	Parses arguments, coordinates both modes, measures time, and frees memory.	Resource management and evaluation.

5.5 Input and Output Design

Input format: ./parallel_search <directory> <keyword> <threads>. For example, ./parallel_search test_data error 4 searches the text files in test_data for the case-insensitive substring “error” using four workers in the parallel phase.

For every matching line, the output includes the worker identifier, filename, line number, and the line contents. The final summary presents sequential and parallel matching-file and matching-line totals, elapsed times, calculated speedup, and a correctness message indicating whether the counts agree.

CHAPTER 6 - IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is intentionally organized as a single C source file for straightforward compilation during a live hackathon demo. The functions are nevertheless separated by responsibility so the logic can be explained and extended independently.

- Input validation: confirms that the keyword is non-empty and the requested thread count is positive.
- Directory scan: creates a dynamic array containing paths of regular files in the target directory.
- Search engine: scans each file in text mode and checks each line using strcasestr().
- Worker management: creates a fixed number of pthread workers and waits for all created workers.
- Statistics: records counts locally per worker and aggregates them after joins.
- Cleanup: frees the filename list and worker arrays, then destroys mutexes.

6.2 Algorithm / Pseudocode

Algorithm: Parallel File Search

1. Read directory, keyword, and thread_count.
2. If arguments are invalid, display usage/error and stop.
3. Enumerate regular files into files[].
4. Start timer; sequentially call search_file() for each file; stop timer.
5. Set next_file = 0 and create thread_count workers.
6. Each worker repeatedly claims one file index under task_mutex and searches that file.
7. Main thread joins the workers.
8. Add each worker's local counters to the parallel totals.
9. Compare sequential and parallel counters; display elapsed times and speedup.
10. Free allocated memory and destroy mutexes.

6.3 Core Implementation Details

Dynamic assignment is implemented through a shared next_file integer. The mutex critical section is deliberately short to prevent workers from waiting while an entire file is being scanned. Output is printed under a separate lock to keep each match readable. A single worker may process more files than another if the file list or scan times differ; dynamic claiming helps distribute such uneven work.

The speedup formula used is:

Speedup = Sequential elapsed time / Parallel elapsed time

A value greater than 1 indicates that the parallel phase was faster in that run. A value less than 1 indicates overhead exceeded the benefit. The result is sensitive to dataset size, CPU scheduling, file-system caching, storage, and worker count.

6.4 Linux APIs Used

API / function	Role in the program
opendir(), readdir(), closedir()	Open, enumerate, and close the directory stream.
stat(), S_ISREG()	Check whether a path identifies a regular file.
fopen(), fgets(), ferror(), fclose()	Open a file, read its lines, detect read errors, and close the file.
pthread_create(), pthread_join()	Start worker threads and wait for their completion.
pthread_mutex_lock(), pthread_mutex_unlock()	Protect task assignment and console output.
clock_gettime(CLOCK_MONOTONIC)	Measure elapsed durations using a monotonic clock.
realloc(), strdup(), free()	Allocate, grow, duplicate, and release the file-path list.

6.5 Process / Thread Implementation

One operating-system process runs the application. During the parallel phase, the process starts up to the requested number of POSIX worker threads. Threads share the file list and synchronization objects but each ThreadData instance is assigned to one worker and stores that worker's statistics. The main thread does not print final totals until after pthread_join() has returned for every created worker.

6.6 IPC / Synchronization Implementation

Inter-process communication is not required because all workers belong to the same process and communicate through shared memory. Synchronization is implemented with pthread mutexes. task_mutex makes file-index allocation atomic; output_mutex prevents output lines from overlapping. Per-thread statistics reduce contention by avoiding a shared counter update on every match.

6.7 Memory Management Implementation

The file list grows using realloc() each time a path is added, and strdup() stores a copy of each path. The thread and thread-data arrays are allocated before worker creation. At the end of the program, every duplicated path is freed, followed by the path array itself. The worker arrays are freed inside parallel_search(), and initialized mutexes are destroyed before program exit.

6.8 Optional User Interface Implementation

The current version uses a terminal interface to keep the project focused on OS concepts rather than graphical development. It prints clear banners for the sequential and parallel phases, reports each matching line, and provides a final summary. Future versions could add a graphical interface or JSON output without changing the underlying thread worker model.

CHAPTER 7 - EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open a terminal in the project directory and install the build tools if necessary:

```
sudo apt update
sudo apt install build-essential
```

Compile the source with:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 parallel_search.c -o parallel_search -pthread
```

A successful compilation returns to the shell prompt without compiler errors and creates the executable named parallel_search.



Figure 7.1. Successful GCC build and executable check in a terminal-style capture.

7.2 Execution Commands

The command syntax is:

```
./parallel_search <directory> <keyword> <threads>
```

Example commands:

```
./parallel_search test_data error 4
./parallel_search test_data error 2
./parallel_search benchmark_data CRITICAL 4
```

7.3 Sample Inputs and Outputs

The five-file sample dataset contains mixed-case variants of the word “error”. The search is case-insensitive, so the line “Second ERROR in log” is included. The sequential and four-thread phases both report three matching files and four matching lines, which satisfies the correctness criterion for this test.

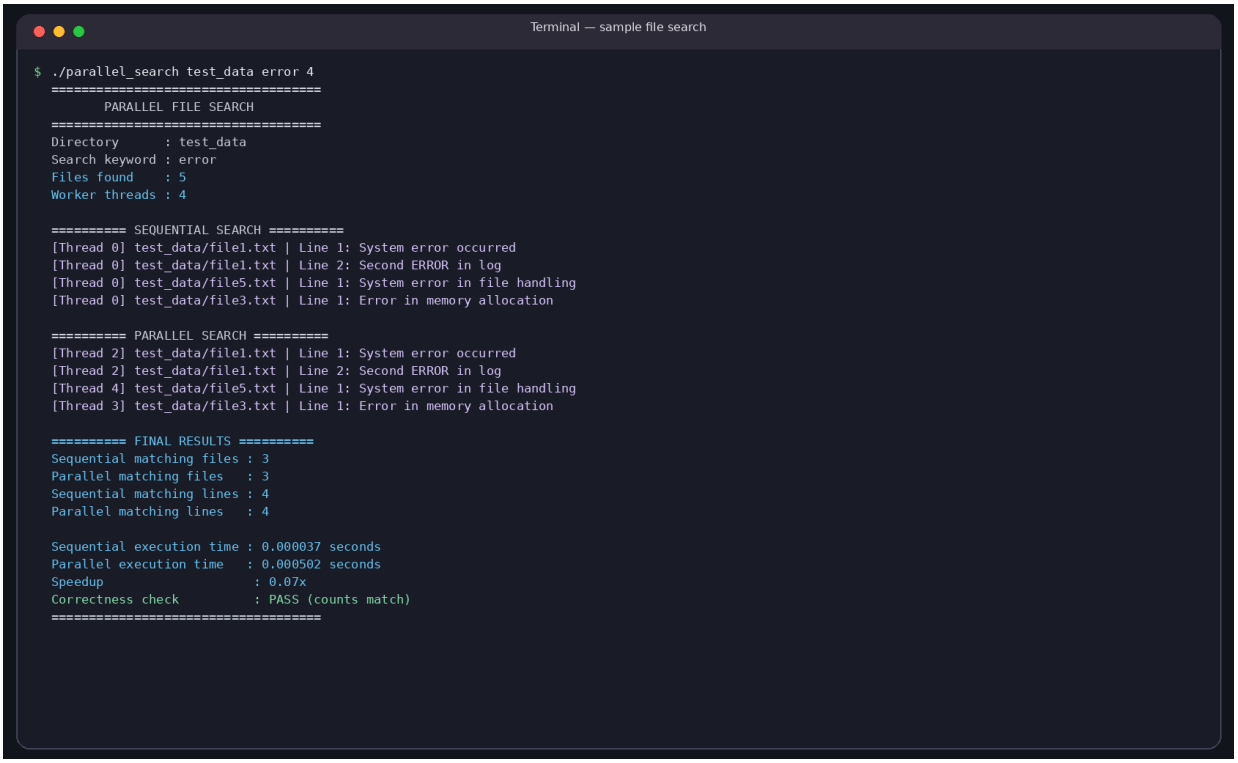


Figure 7.2. Sample run showing sequential and four-thread results for five test files.

7.4 Screenshots of Execution

The following capture shows the benchmark summary from a synthetic workload containing 300 files, each with 350 lines (approximately 105,000 lines overall). The keyword CRITICAL appears in two lines in each of 12 files, so the expected totals are 12 matching files and 24 matching lines.

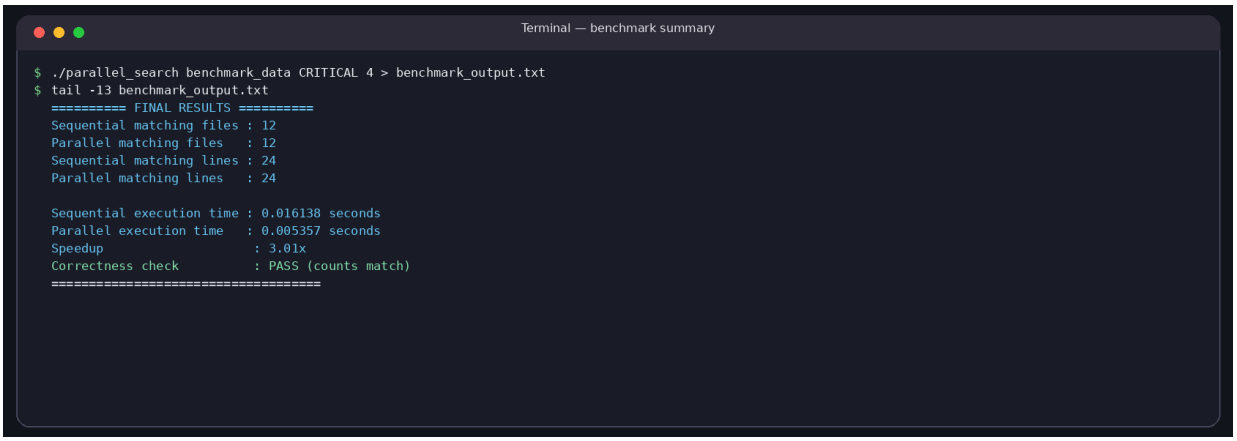


Figure 7.3. Recorded benchmark summary; timing values represent one run in the report-generation Linux environment.

7.5 Demonstration of OS Concepts

• Thread creation: pthread_create() starts the worker routines.

• Thread completion: pthread_join() prevents the main thread from aggregating incomplete statistics.

• Mutex synchronization: one mutex protects shared task allocation and a second protects the displayed output.

• Shared address space: workers access the same file-path array and keyword while maintaining separate local statistics.

• Performance evaluation: sequential and parallel elapsed times are measured and compared for the same dataset.

CHAPTER 8 - TESTING, RESULTS AND IMPACT

8.1 Testing Methodology

Testing covers functional correctness, mixed-case matching, no-match behavior, empty directories, invalid worker counts, and consistency across different thread counts. For each ordinary search case, the sequential and parallel matching-file and matching-line totals are compared. Timing measurements are recorded but are not used as the sole criterion for correctness.

8.2 Test Cases and Outcomes

ID	Test input / scenario	Expected result	Observed outcome
T1	5 files; keyword error; 4 workers	3 matching files; 4 matching lines; sequential/parallel totals agree.	PASS - 3 files and 4 lines in both modes.
T2	Mixed case: error / ERROR / Error	Matching is case-insensitive.	PASS - all four matching lines found.
T3	5 files; keyword banana; 4 workers	0 matching files and 0 matching lines.	PASS - both totals were zero.
T4	Run sample with 1, 2, and 4 workers	Counts remain identical for each thread count.	PASS - sequential/parallel counts agreed in the recorded runs.
T5	Empty directory	Inform the user that no regular files were found.	PASS - message displayed.
T6	Thread count 0	Reject invalid input.	PASS - input validation message displayed.
T7	300 files × 350 lines; keyword CRITICAL; 4 workers	12 files / 24 lines in both modes.	PASS - totals agreed; recorded speedup 3.01×.

8.3 Results Achieved

Correctness was the primary criterion: all recorded runs produced matching sequential and parallel totals. The five-file smoke test correctly found three matching files and four matching lines. The no-match case returned zero matches, the empty-directory case exited with a clear message, and invalid thread count input was rejected.

For the synthetic benchmark, the program scanned 300 files containing approximately 105,000 lines. The observed run reported 12 matching files and 24 matching lines in both modes; sequential elapsed time was 0.016138 seconds, parallel elapsed time was 0.005357 seconds, and the calculated speedup was 3.01×. This is a single captured run in the report-generation Linux environment. The numbers can differ on the student's Ubuntu machine and should be re-measured locally before a formal performance claim.

8.4 Performance Interpretation

The small five-file test showed that parallel execution was slower than sequential execution because thread creation, scheduling, locking, and joining can cost more time than scanning a few short files. The larger synthetic dataset provided enough independent file work to show an observed speedup in the captured run. This illustrates why the performance benefit depends on workload size and system conditions rather than thread count alone.

8.5 Impact and Applications

• Log scanning: locate recurring errors or warning messages across multiple log files.

• Source-code search: find identifiers or suspicious strings in files directly within a directory.

• Document repositories: scan text exports or plain-text documentation for matching phrases.

• OS education: make threads, mutexes, race-condition prevention, thread joins, and benchmark variation observable.

• Extendability: provides a foundation for file filters, recursive traversal, a progress indicator, or machine-readable results.

8.6 Current Limitations and Future Enhancements

• Current scope is non-recursive and targets regular text files; the 4096-byte line buffer and byte-wise matching limit handling of very long lines, binary files, and complex encodings.

• Future work includes recursive traversal, file filters, robust long-line processing, progress reporting, machine-readable output, and repeatable benchmark trials with documented hardware and dataset conditions.

CHAPTER 9 - CONCLUSION AND REFERENCES

9.1 Summary of the Project

Parallel File Search Using Threads is a command-line C application that demonstrates how a directory of independent text files can be searched with POSIX worker threads. It uses a shared task index, mutexes, per-thread result counters, and pthread joins to coordinate the work. The program runs a sequential baseline for comparison and reports matching lines, matching files, elapsed time, and speedup.

9.2 Objectives Achieved

- Implemented command-line keyword search across regular files in a directory.
- Added sequential and configurable pthread-based parallel search phases.
- Used mutexes for safe work allocation and readable console output.
- Reported filename and line number for each match and aggregated per-thread statistics.
- Validated result consistency on positive, no-match, empty-directory, invalid-input, and larger synthetic tests.
- Collected one larger-workload timing result and documented its machine/workload dependence.

9.3 Skills and Learning Outcomes

The project reinforces practical knowledge of the Linux C toolchain, directory and file APIs, POSIX thread lifecycle, mutex synchronization, dynamic memory management, elapsed-time measurement, test-case design, and the interpretation of performance results. It also demonstrates the value of starting from a correct sequential baseline before adding concurrency.

9.4 References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.

[2] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, No Starch Press, 2010.

[3] The Open Group, POSIX.1 / IEEE Std 1003.1, pthreads and thread synchronization interfaces.

[4] GNU Compiler Collection (GCC) documentation, compiler options and pthread support: https://gcc.gnu.org/onlinedocs/

[5] Linux man-pages project: pthread_create(3), pthread_join(3), pthread_mutex_lock(3), opendir(3), readdir(3), clock_gettime(3), fgets(3), and strcasestr(3). https://man7.org/linux/man-pages/

APPENDIX A - FULL C SOURCE CODE

The listing below is the source used for the recorded test runs. Save it as parallel_search.c and compile it with the command in Chapter 7.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <pthread.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>

#define MAX_PATH_LEN 4096

char **files = NULL;
int file_count = 0;
const char *keyword = NULL;
pthread_mutex_t task_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t output_mutex = PTHREAD_MUTEX_INITIALIZER;
int next_file = 0;

typedef struct {
    long matching_lines;
    long matching_files;
} SearchStats;

typedef struct {
    int thread_id;
    SearchStats stats;
} ThreadData;

static double get_time(void) {
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        perror("clock_gettime");
        exit(EXIT_FAILURE);
    }
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1000000000.0;
}

static void add_file(const char *path) {
    char **temp = realloc(files, (size_t)(file_count + 1) * sizeof(*files));
    if (temp == NULL) {
        perror("realloc");
        exit(EXIT_FAILURE);
    }
    files = temp;
    files[file_count] = strdup(path);
    if (files[file_count] == NULL) {
        perror("strdup");
        exit(EXIT_FAILURE);
    }
    file_count++;
}

static void collect_files(const char *directory) {
    DIR *dir = opendir(directory);
    if (dir == NULL) {
        perror("opendir");
        exit(EXIT_FAILURE);
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] == '.') {
            continue;
        }

        char path[MAX_PATH_LEN];
        int written = snprintf(path, sizeof(path), "%s/%s", directory, entry->d_name);
        if (written < 0 || (size_t)written >= sizeof(path)) {
            fprintf(stderr, "Skipping path that is too long: %s\n", entry->d_name);
            continue;
```

```
        }

        struct stat info;
        if (stat(path, &info) == 0 && S_ISREG(info.st_mode)) {
            add_file(path);
        }
    }
    closedir(dir);
}

static SearchStats search_file(const char *filename, int thread_id) {
    SearchStats stats = {0, 0};
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        pthread_mutex_lock(&output_mutex);
        fprintf(stderr, "Unable to open %s: ", filename);
        perror("");
        pthread_mutex_unlock(&output_mutex);
        return stats;
    }

    char line[4096];
    int line_number = 0;
    int matched_this_file = 0;
    while (fgets(line, sizeof(line), fp) != NULL) {
        line_number++;
        if (strcasestr(line, keyword) != NULL) {
            stats.matching_lines++;
            matched_this_file = 1;
            pthread_mutex_lock(&output_mutex);
            printf("[Thread %d] %s | Line %d: %s", thread_id, filename, line_number, line);
            if (strchr(line, '\n') == NULL) {
                putchar('\n');
            }
            pthread_mutex_unlock(&output_mutex);
        }
    }
    if (ferror(fp)) {
        pthread_mutex_lock(&output_mutex);
        fprintf(stderr, "Read error in %s\n", filename);
        pthread_mutex_unlock(&output_mutex);
    }
    fclose(fp);

    if (matched_this_file) {
        stats.matching_files++;
    }
    return stats;
}

static void add_stats(SearchStats *total, SearchStats current) {
    total->matching_lines += current.matching_lines;
    total->matching_files += current.matching_files;
}

static SearchStats sequential_search(void) {
    SearchStats total = {0, 0};
    printf("\n========== SEQUENTIAL SEARCH ==========\n");
    for (int i = 0; i < file_count; i++) {
        add_stats(&total, search_file(files[i], 0));
    }
    return total;
}

static void *worker(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    for (;;) {
        int task;
        pthread_mutex_lock(&task_mutex);
        if (next_file >= file_count) {
            pthread_mutex_unlock(&task_mutex);
            break;
        }
        task = next_file++;
        pthread_mutex_unlock(&task_mutex);

        add_stats(&data->stats, search_file(files[task], data->thread_id));
    }
    return NULL;
```

```
}

static SearchStats parallel_search(int thread_count) {
    printf("\n========== PARALLEL SEARCH ==========\n");
    next_file = 0;

    pthread_t *threads = calloc((size_t)thread_count, sizeof(*threads));
    ThreadData *data = calloc((size_t)thread_count, sizeof(*data));
    if (threads == NULL || data == NULL) {
        perror("calloc");
        free(threads);
        free(data);
        exit(EXIT_FAILURE);
    }

    int created = 0;
    for (int i = 0; i < thread_count; i++) {
        data[i].thread_id = i + 1;
        if (pthread_create(&threads[i], NULL, worker, &data[i]) != 0) {
            fprintf(stderr, "pthread_create failed for worker %d\n", i + 1);
            break;
        }
        created++;
    }

    for (int i = 0; i < created; i++) {
        if (pthread_join(threads[i], NULL) != 0) {
            fprintf(stderr, "pthread_join failed for worker %d\n", i + 1);
            free(threads);
            free(data);
            exit(EXIT_FAILURE);
        }
    }

    SearchStats total = {0, 0};
    for (int i = 0; i < created; i++) {
        add_stats(&total, data[i].stats);
    }
    free(threads);
    free(data);
    return total;
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Usage: %s <directory> <keyword> <threads>\n", argv[0]);
        fprintf(stderr, "Example: %s test_data error 4\n", argv[0]);
        return EXIT_FAILURE;
    }

    int thread_count = atoi(argv[3]);
    if (argv[2][0] == '\0' || thread_count <= 0) {
        fprintf(stderr, "Keyword must not be empty and thread count must be positive.\n");
        return EXIT_FAILURE;
    }
    keyword = argv[2];

    collect_files(argv[1]);
    if (file_count == 0) {
        printf("No regular files found in the directory.\n");
        free(files);
        return EXIT_SUCCESS;
    }

    printf("======================================\n");
    printf("       PARALLEL FILE SEARCH\n");
    printf("======================================\n");
    printf("Directory      : %s\n", argv[1]);
    printf("Search keyword : %s\n", keyword);
    printf("Files found    : %d\n", file_count);
    printf("Worker threads : %d\n", thread_count);

    double start = get_time();
    SearchStats seq_stats = sequential_search();
    double sequential_time = get_time() - start;

    start = get_time();
```

```
    SearchStats par_stats = parallel_search(thread_count);
    double parallel_time = get_time() - start;

    printf("\n========== FINAL RESULTS ==========\n");
    printf("Sequential matching files : %ld\n", seq_stats.matching_files);
    printf("Parallel matching files   : %ld\n", par_stats.matching_files);
    printf("Sequential matching lines : %ld\n", seq_stats.matching_lines);
    printf("Parallel matching lines   : %ld\n", par_stats.matching_lines);
    printf("\nSequential execution time : %.6f seconds\n", sequential_time);
    printf("Parallel execution time   : %.6f seconds\n", parallel_time);
    if (parallel_time > 0.0) {
        printf("Speedup                   : %.2fx\n", sequential_time / parallel_time);
    }
    if (seq_stats.matching_lines != par_stats.matching_lines ||
        seq_stats.matching_files != par_stats.matching_files) {
        fprintf(stderr, "ERROR: sequential and parallel result counts differ.\n");
    } else {
        printf("Correctness check         : PASS (counts match)\n");
    }
    printf("====================================\n");

    for (int i = 0; i < file_count; i++) {
        free(files[i]);
    }
    free(files);
    pthread_mutex_destroy(&task_mutex);
    pthread_mutex_destroy(&output_mutex);
    return (seq_stats.matching_lines == par_stats.matching_lines &&
            seq_stats.matching_files == par_stats.matching_files)
              ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

APPENDIX B - REPRODUCIBLE TEST COMMANDS

From the directory containing parallel_search.c, run:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 parallel_search.c -o parallel_search -pthread
./parallel_search test_data error 4
./parallel_search test_data banana 4
./parallel_search benchmark_data CRITICAL 4
```

The supplied project archive contains a five-file functional test dataset and a generated benchmark dataset. For submission, the student should run these commands on their own Ubuntu installation and replace any environment-specific timing values if they differ.